

JavaScript-Überblick: Grundlegende Funktionen und Methoden

Entdecken Sie die Vielfalt und Leistungsfähigkeit von JavaScript-Funktionen und Methoden für die moderne Webentwicklung.

Jetzt erkunden

Beispiele ansehen

getElementById(): Elemente gezielt ansprechen

Funktionsbeschreibung

Die Methode getElementById() ermöglicht es, ein spezifisches Element anhand seiner ID zu selektieren. Dies ist besonders nützlich, um einzelne Elemente gezielt zu manipulieren oder Ereignisse an sie zu binden.

Beispielcode

document.getElementById("demo").innerText = "Hallo!";

Anwendungsbeispiel

Erstellen Sie eine Webseite mit einem div-Element und ändern Sie dessen Text mit getElementById zu "Hallo Welt!".

querySelector(): Flexible Elementauswahl

1 Funktionsbeschreibung

querySelector() wählt das erste Element aus, das dem angegebenen CSS-Selektor entspricht. Dies ermöglicht eine flexible und präzise Auswahl von Elementen.

2 Beispielcode

```
document.querySelector(".demo-class").style.color = "blue";
```

3 Anwendungsbeispiel

Entwickeln Sie eine HTML-Seite mit einem Element der Klasse demo-class und ändern Sie dessen Farbe mit querySelector auf Blau.



addEventListener(): Interaktivität hinzufügen

Funktionsbeschreibung

addEventListener() fügt einem Element einen Ereignis-Listener hinzu, der auf bestimmte Aktionen reagiert. Dies ist grundlegend für die Erstellung interaktiver Webseiten.

Beispielcode

```
document.getElementById("btn").
addEventListener("click",
function() { alert("Geklickt!"); });
```

Anwendungsbeispiel

Erstellen Sie eine einfache HTML-Seite mit einem Button, der beim Klicken eine Alert-Box anzeigt, implementiert mit addEventListener.

setTimeout(): Verzögerte Ausführung

Funktionsbeschreibung

1

setTimeout() führt Code nach einer festgelegten Zeitverzögerung aus. Dies ist nützlich für die Implementierung von Verzögerungen oder periodischen Aktionen.

Beispielcode

2

setTimeout(function() { alert("Zeit abgelaufen!"); }, 3000);

Anwendungsbeispiel

3

Erstellen Sie eine HTML-Seite, die nach drei Sekunden eine Benachrichtigung anzeigt, realisiert durch setTimeout.

fetch(): Netzwerkanfragen leicht gemacht

1

Funktionsbeschreibung

fetch() ermöglicht das Senden von Netzwerkanfragen und den Abruf von Ressourcen. Es ist besonders nützlich für die Arbeit mit APIs und den Datenaustausch.

2

Beispielcode

fetch('https://api.example.com/data').then(response => response.json()).then(data => console.log(data));

3

Anwendungsbeispiel

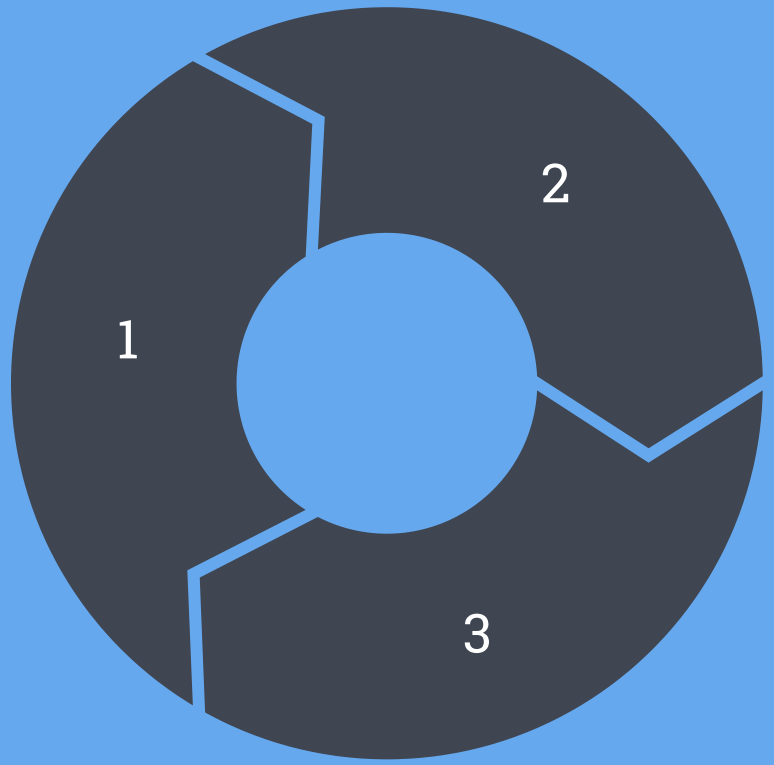
Entwickeln Sie eine Web-App, die Daten von einer API abruft und diese im Browser anzeigt, unter Verwendung von fetch.



Array.forEach(): Iteration leicht gemacht

Funktionsbeschreibung

Array.forEach() führt eine Funktion für jedes Element eines Arrays aus. Es vereinfacht die Iteration über Arrayelemente.



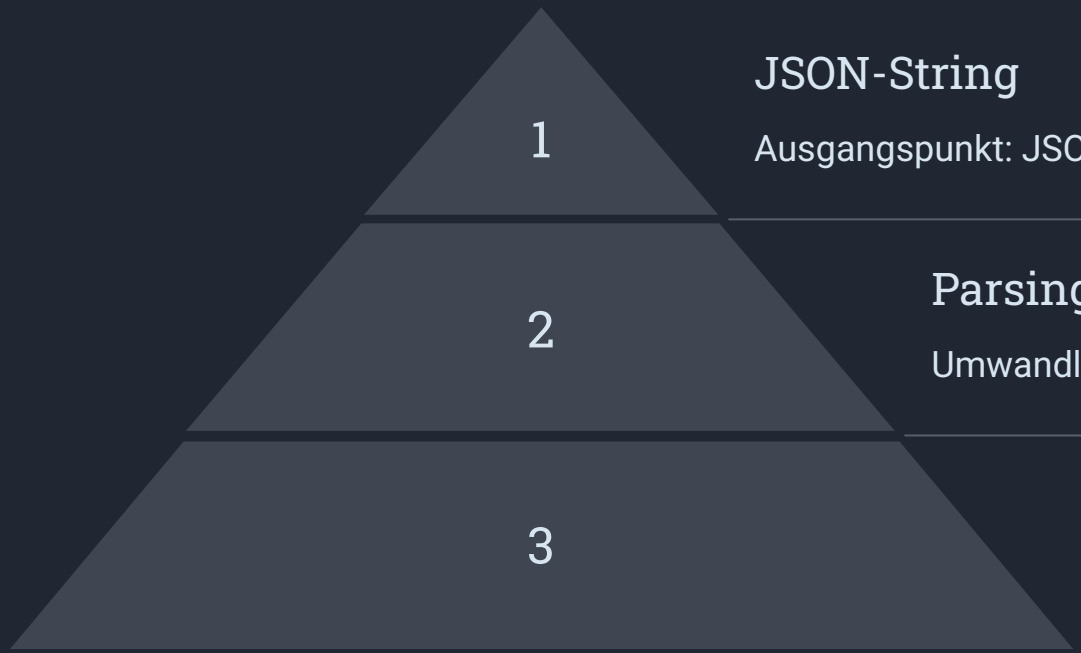
Beispielcode

```
[1, 2, 3].forEach(num =>
console.log(num));
```

Anwendungsbeispiel

Schreiben Sie ein Skript, das ein Array durchläuft und jeden Wert in der Konsole ausgibt, unter Verwendung von Array.forEach.

JSON.parse(): JSON in JavaScript-Objekte umwandeln



JSON-String

Ausgangspunkt: JSON-formatierter String

Parsing-Prozess

Umwandlung durch JSON.parse()

JavaScript-Objekt

Resultat: Nutzbares JS-Objekt

JSON.stringify(): JavaScript-Objekte in JSON umwandeln

1

JavaScript-Objekt

Ausgangspunkt: JS-Objekt

2

Stringifizierung

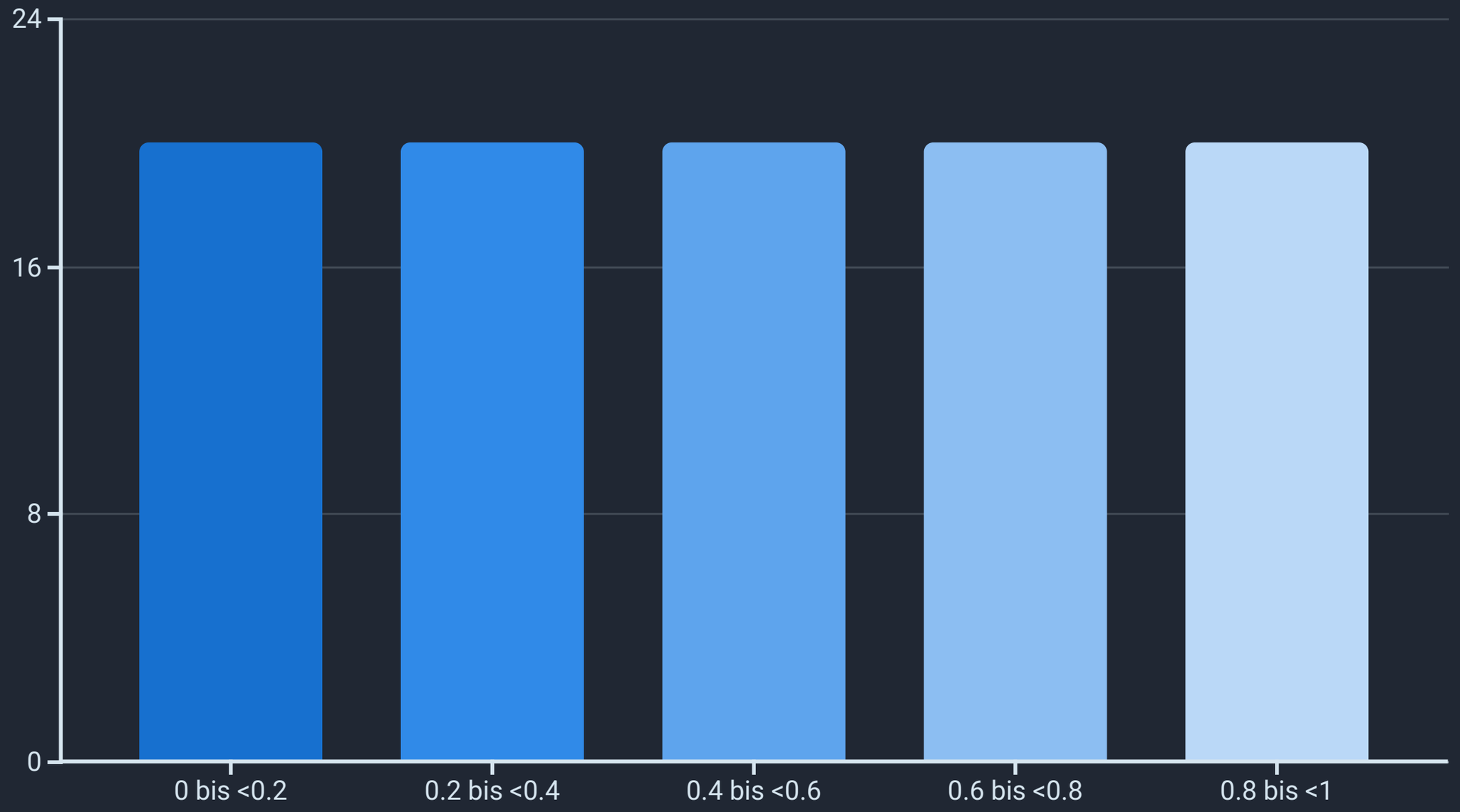
Umwandlung durch JSON.stringify()

3

JSON-String

Resultat: JSON-formatierter String

Math.random(): Zufallszahlen generieren



Math.random() erzeugt eine Zufallszahl zwischen 0 (inklusive) und 1 (exklusive). Dies ist nützlich für Spiele, Simulationen oder zufällige Auswahlen.

Array.map(): Transformation von Arrays

1

Originales Array

Ausgangspunkt für die Transformation

2

Mapping-Funktion

Definiert die Transformation

3

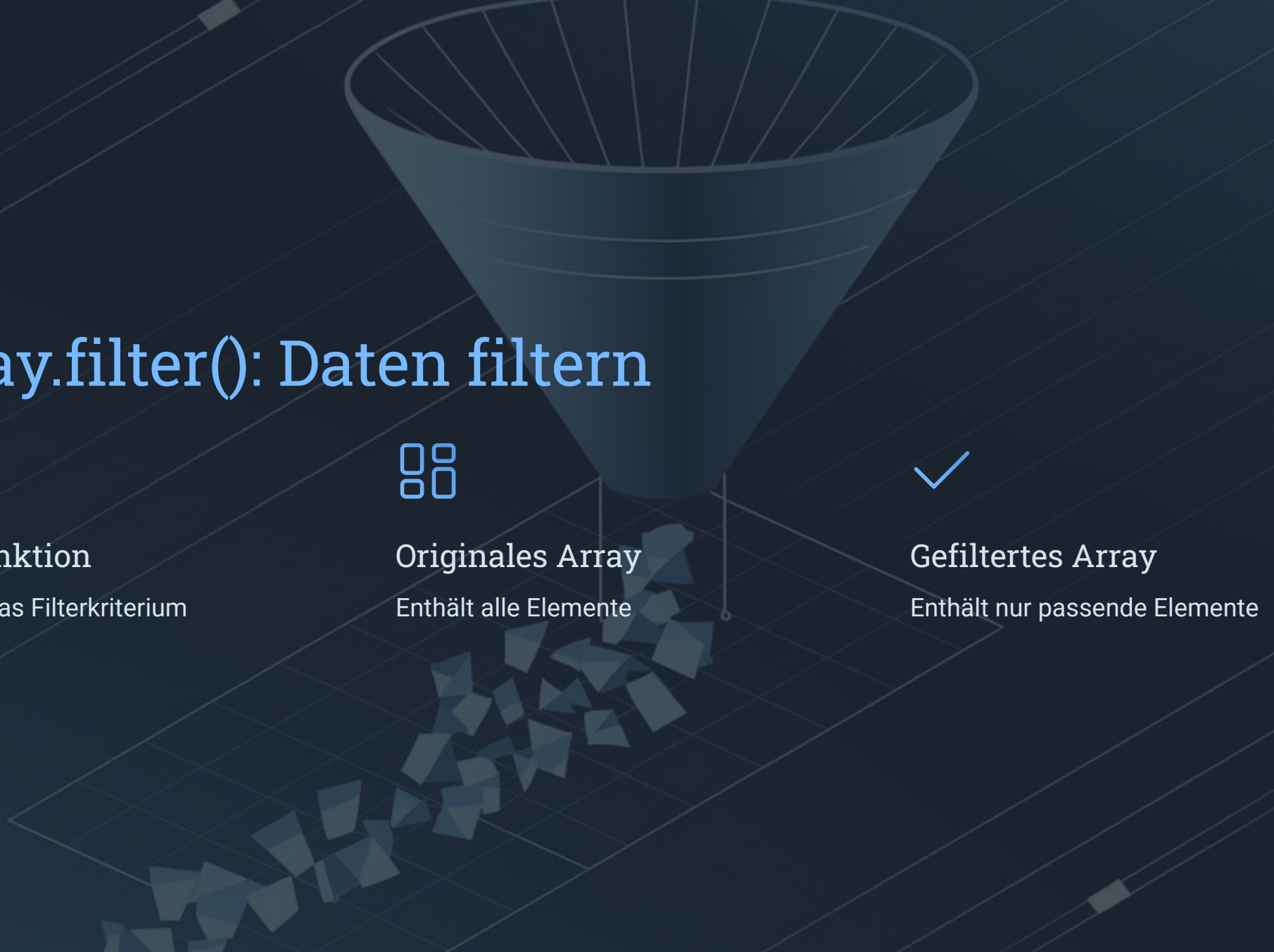
Neues Array

Ergebnis der Transformation

Array.map() erstellt ein neues Array mit den Ergebnissen des Aufrufs einer bereitgestellten Funktion auf jedem Element im aufrufenden Array.



Array.filter(): Daten filtern



Filterfunktion

Definiert das Filterkriterium

Originales Array

Enthält alle Elemente

Gefiltertes Array

Enthält nur passende Elemente

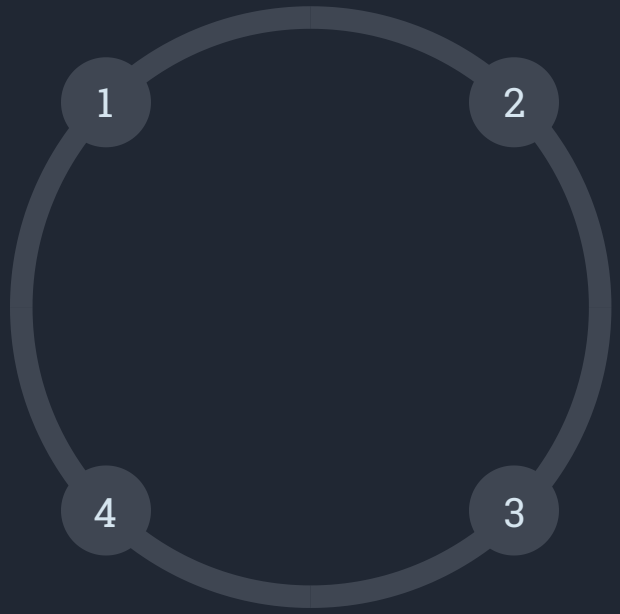
Array.reduce(): Arrays auf einen Wert reduzieren

Initialer Wert

Startwert für die Reduktion

Reduktionsfunktion

Definiert die Reduktionslogik



Finaler Wert

Ergebnis der Reduktion

Array-Elemente

Werden schrittweise verarbeitet

String.replace(): Zeichenketten manipulieren

Originaler String

Ausgangstext für die Manipulation

Suchmuster

Zu ersetzender Text oder regulärer Ausdruck

Ersetzung

Neuer Text oder Funktion zur Generierung des neuen Textes

Date.now(): Aktuelle Zeit in Millisekunden

1 ## Funktionsbeschreibung

Date.now() gibt die Anzahl der Millisekunden zurück, die seit dem 1. Januar 1970 00:00:00 UTC vergangen sind.

2 ## Anwendungsbeispiel

Implementieren Sie ein Skript, das den aktuellen Zeitstempel in Millisekunden anzeigt.

3 ## Nutzungsmöglichkeiten

Zeitmessung, Erstellung eindeutiger IDs, Leistungsmessung von Code

Object.keys(): Objekteigenschaften auflisten

Funktionsbeschreibung

Object.keys() gibt ein Array mit den Namen der aufzählbaren Eigenschaften eines Objekts zurück.

Beispielcode

```
let keys = Object.keys({a: 1, b: 2});
```

Anwendungsbeispiel

Erstellen Sie ein Skript, das alle Eigenschaftsnamen eines Objekts auflistet.



Object.values(): Objektwerte auflisten

1

Funktionsbeschreibung

Object.values() gibt ein Array mit den Werten der aufzählbaren Eigenschaften eines Objekts zurück.

2

Beispielcode

let values = Object.values({a: 1, b: 2});

3

Anwendungsbeispiel

Implementieren Sie ein Skript, das alle Eigenschaftswerte eines Objekts auflistet.

Object.assign(): Objekte zusammenführen

1

Zielobjekt

Das Objekt, in das die Eigenschaften kopiert werden

2

Quellobjekte

Ein oder mehrere Objekte, deren Eigenschaften kopiert werden

3

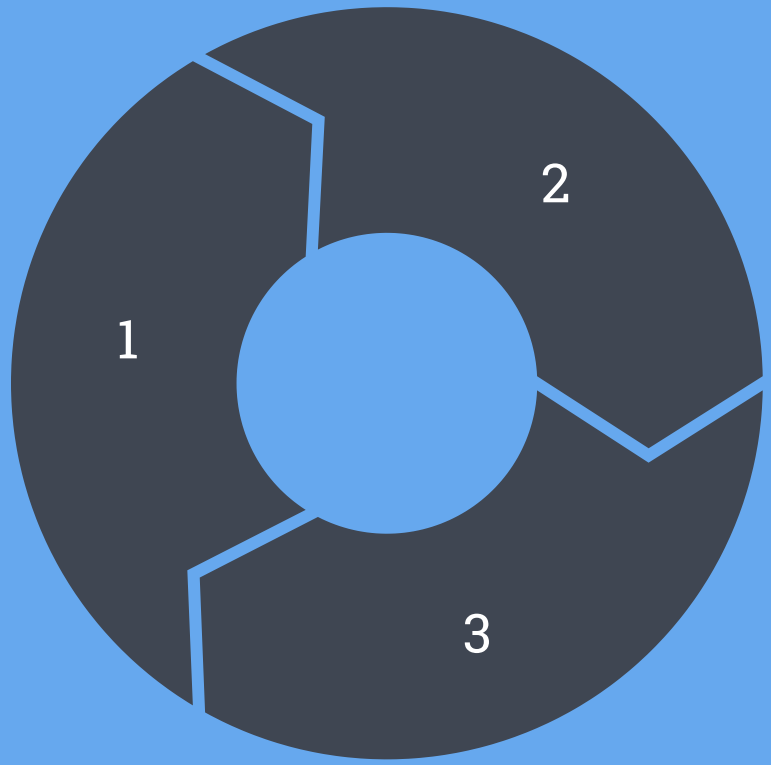
Ergebnisobjekt

Das modifizierte Zielobjekt mit allen kopierten Eigenschaften

Math.floor(): Abrunden von Zahlen

Eingabe

Beliebige Dezimalzahl



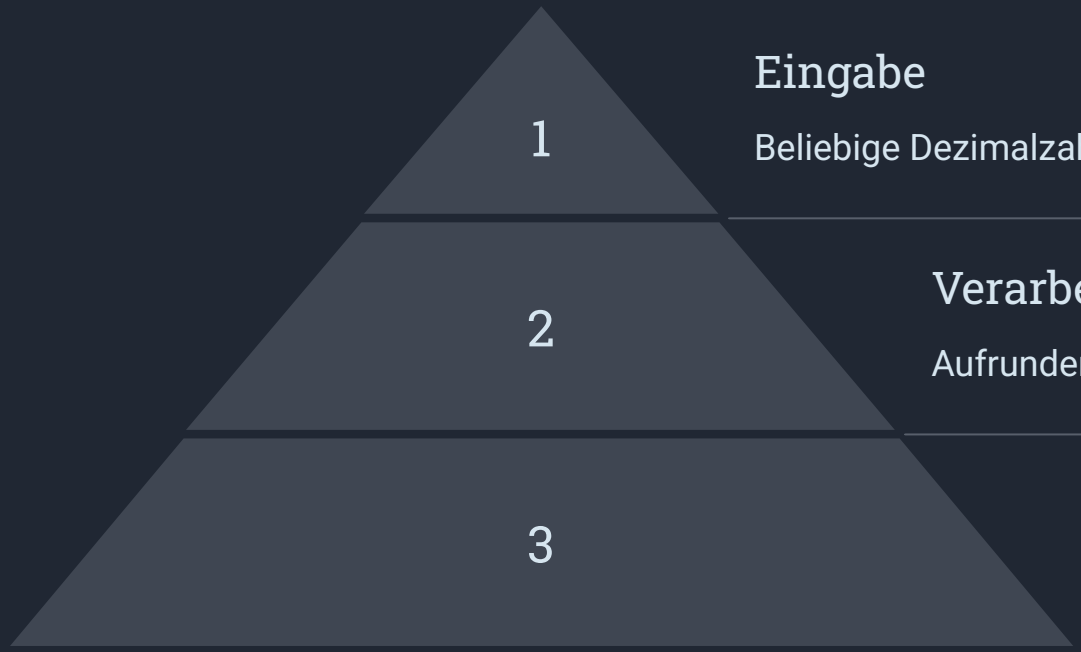
Verarbeitung

Abrunden zur nächsten ganzen Zahl

Ausgabe

Nächstkleinere ganze Zahl

Math.ceil(): Aufrunden von Zahlen



Eingabe

Beliebige Dezimalzahl

Verarbeitung

Aufrunden zur nächsten ganzen Zahl

Ausgabe

Nächstgrößere ganze Zahl

Array.some(): Prüfen von Arraybedingungen

1

Array

Ausgangspunkt für die Prüfung

2

Prüffunktion

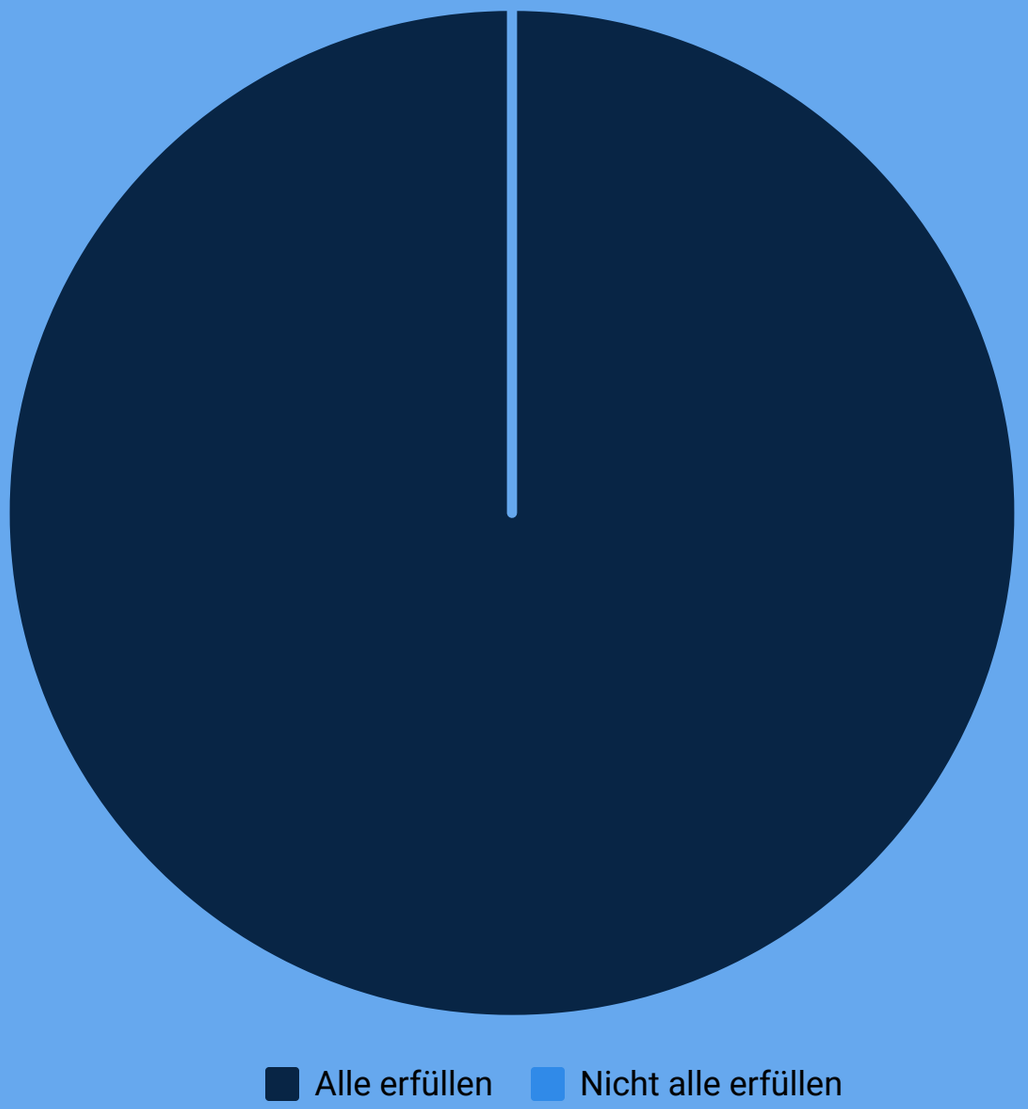
Definiert die zu prüfende Bedingung

3

Ergebnis

true, wenn mindestens ein Element die Bedingung erfüllt

Array.every(): Universelle Arrayprüfung



Array.every() prüft, ob alle Elemente eines Arrays eine bestimmte Bedingung erfüllen. Es gibt true zurück, wenn die Bedingung für alle Elemente zutrifft, sonst false.

String.indexOf(): Textsuche in Strings

0

Startindex

Beginn der Suche

-1

Nicht gefunden

Rückgabewert bei erfolgloser Suche

n

Fundstelle

Index des ersten Vorkommens



String.slice(): Teilstrings extrahieren



Originalstring

Ausgangstext für die Extraktion



Slice-Operation

Extraktion des gewünschten Teils



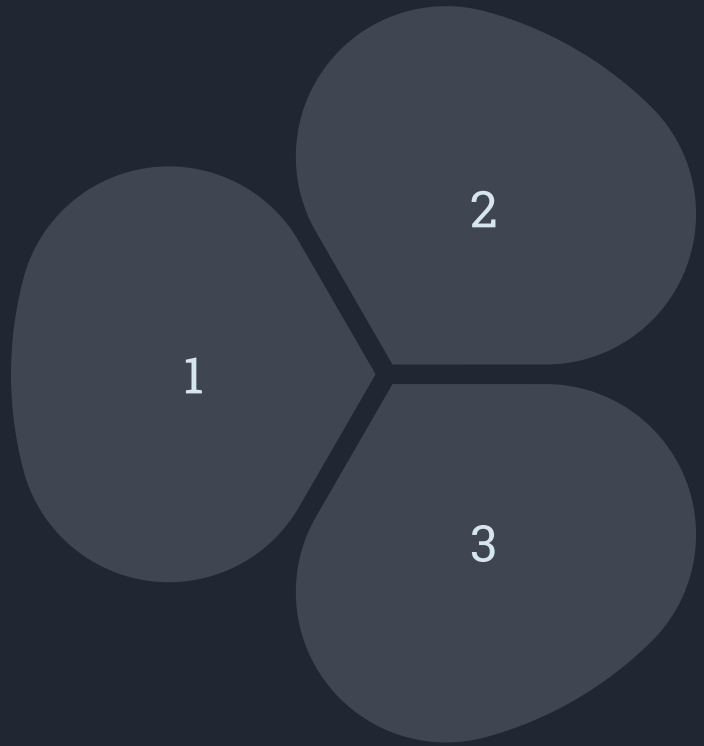
Extrahierter String

Ergebnis der Slice-Operation

String.toUpperCase(): Text in Großbuchstaben umwandeln

Eingabetext

Originaltext in beliebiger Schreibweise



Umwandlung

Anwendung von toUpperCase()

Ergebnis

Text komplett in Großbuchstaben

String.toLowerCase(): Text in Kleinbuchstaben umwandeln

Eingabetext

Originaltext in beliebiger Schreibweise

Umwandlung

Anwendung von toLowerCase()

Ergebnis

Text komplett in Kleinbuchstaben

Number.isNaN(): Überprüfung auf NaN

1. ## Funktionsbeschreibung

   Number.isNaN() prüft, ob ein Wert NaN (Not-a-Number) ist.

2. ## Beispielcode

   Number.isNaN(NaN); // gibt true zurück

3. ## Anwendungsbeispiel

   Erstellen Sie ein Skript, das prüft, ob ein Wert NaN ist.

Number.isInteger(): Ganzzahlenprüfung

Funktionsbeschreibung

Number.isInteger() bestimmt, ob der übergebene Wert eine ganze Zahl ist.

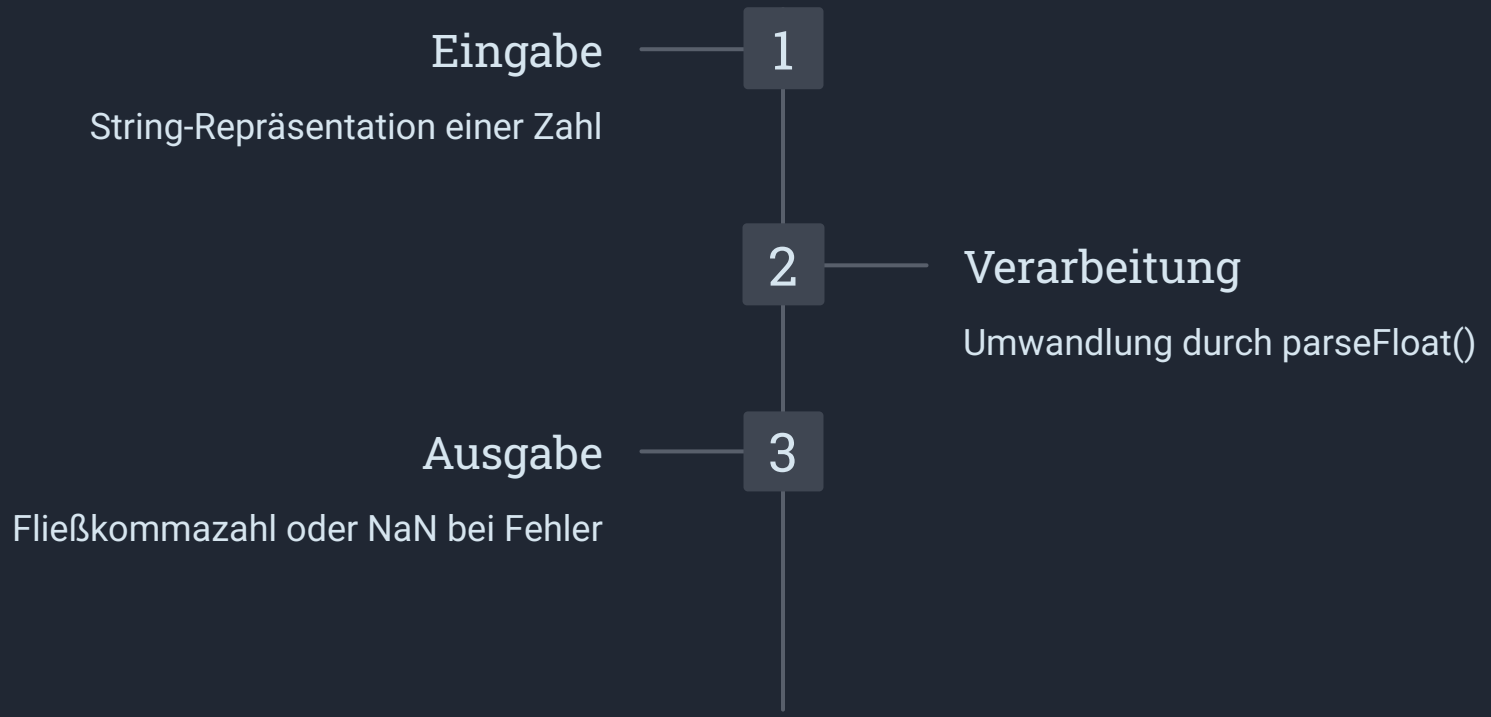
Beispielcode

Number.isInteger(4); // gibt true zurück

Anwendungsbeispiel

Implementieren Sie ein Skript, das prüft, ob ein Wert eine ganze Zahl ist.

Number.parseFloat(): String zu Fließkommazahl



Number.parseInt(): String zu Ganzzahl

1

Eingabe

String-Repräsentation einer Zahl

2

Verarbeitung

Umwandlung durch parseInt()

3

Ausgabe

Ganzzahl oder NaN bei Fehler